

MANUAL DE DESARROLLO VISUAL

Dashboard *Streamlit.*

Cómo se diseñó y construyó el dashboard interactivo: sistema de diseño, tokens, inyección de CSS, componentes, visualizaciones y decisiones de UX.

11

CAPÍTULOS

5

PÁGINAS APP

2

FUENTES

18

TIPOS COLOR

Contenido

Once capítulos para entender el dashboard de principio a fin: desde por qué se eligió Streamlit hasta cómo personalizarlo y extenderlo con nuevas páginas.

01	¿Por qué Streamlit?	3
	Ventajas, limitaciones y perfil que demuestra	
02	Arquitectura de <code>app.py</code>	4
	Estructura del archivo y flujo de ejecución	
03	Sistema de diseño — Tokens y tipografía	5
	Paleta, tipos Pokémon y jerarquía	
04	Inyección de CSS	6
	Cómo personalizar Streamlit con <code>data-testid</code>	
05	Componentes clave	7
	KPI Card, Type Badge, Hero, Poke Card	
06	Navegación — Sidebar como menú	8
	Radio buttons estilizados y logo en base64	
07	Las 5 páginas del dashboard	9
	Vista General, Estadísticas, Galería, Comparación, ETL	
08	Visualizaciones — Plotly en Streamlit	10
	Express vs Graph Objects, radar y estilos	
09	Manejo de datos — Cache y filtros	11
	<code>@st.cache_data</code> y sincronización	
10	Animaciones y micro-interacciones	11
	<code>@keyframes</code> , hover y transiciones	
11	Cómo ejecutar y personalizar	12
	Comandos, configuración y extensiones	



Cómo leer este manual

Cada capítulo es autocontenido. Los bloques en azul son notas conceptuales, los amarillos advertencias o decisiones de diseño, y los verdes buenas prácticas. El código está siempre dentro de bloques etiquetados con su lenguaje.

Streamlit es un framework de Python que convierte scripts de datos en aplicaciones web sin necesidad de conocimientos de HTML, CSS o JavaScript.

Ventajas para un portafolio de Data Analyst

Criterio	Streamlit	Flask/FastAPI + HTML	Node.js
Curva de aprendizaje	Baja — Python puro	Media — requiere HTML/Jinja	Alta — nuevo lenguaje
Integración con pandas	Nativa	Manual	Manual
Integración con Plotly	Nativa	Manual	Plotly.js
Perfil que demuestra	Data Analyst (ideal)	Data Engineer	Full-Stack
Tiempo de desarrollo	Horas	Días	Semanas



Conclusión

Para un DA, lo que importa es demostrar que dominas el dato y puedes visualizarlo con claridad. Streamlit permite eso sin salir del ecosistema Python.

Limitaciones conocidas de Streamlit

- **Re-render completo** — Cada interacción del usuario ejecuta el script de arriba a abajo (mitigado con `@st.cache_data`).
- **CSS limitado** — No se puede acceder directamente a los componentes renderizados; hay que apuntar a los `data-testid` internos.
- **No es React** — No hay estado local por componente. Todo el estado es global (`st.session_state`).



Implicación práctica

Como cada interacción re-ejecuta el script entero, todo lo costoso (lectura de SQLite, transformaciones) debe estar bajo `@st.cache_data`. Sin cache, mover un slider del sidebar dispararía una consulta SQL completa.

Estructura del archivo

TREE · APP.PY

```
app.py
├── Constantes globales
│   ├── Rutas: DB_PATH, CSV_PATH
│   ├── Tokens de color: ACCENT, BG, CARD_BG, TEXT, TEXT_SUB
│   ├── TYPE_COLORS: paleta por tipo Pokémon
│   └── NAV_PAGES: lista de páginas
├── Funciones de configuración
│   ├── inject_css()      ← Carga fuentes + todo el CSS personalizado
│   └── load_data()       ← Lee datos de SQLite (fallback a CSV)
├── Funciones de UI por página
│   ├── tab_overview()   ← Vista General
│   ├── tab_stats()      ← Estadísticas
│   ├── tab_gallery()    ← Galería
│   ├── tab_compare()    ← Comparación
│   └── tab_etl()         ← Resumen ETL
└── main()
    ├── st.set_page_config()
    ├── inject_css()
    ├── load_data()
    ├── render_sidebar()  ← Logo + filtros + navegación
    └── Router            ← Llama a la función de la página activa
```

Flujo de ejecución

FLUJO

```
Usuario interactúa
├──
└── ▼
    Streamlit re-ejecuta app.py completo
        ├── inject_css()      ← CSS siempre cargado
        ├── load_data()       ← Datos del cache (no re-lee el archivo)
        ├── render_sidebar()  ← Filtros aplicados al DataFrame
        └── tab_*( )          ← Renderiza la página seleccionada
```

i Patrón router

Streamlit no incluye un sistema de rutas; `main()` implementa un router manual con un `if/elif` sobre la variable `page` devuelta por el radio del sidebar. Es simple, explícito y suficiente para una SPA de 5 vistas.

Tokens de color

Los colores no se hardcodean en cada elemento — se definen como constantes al inicio del archivo y se usan mediante f-strings en el CSS.

ACCENT	BG	CARD_BG	TEXT	TEXT_SUB
#2962FF	#F5F7FA	FFFFFF	#111111	#555555

i **Por qué tokens y no hex sueltos**

Si se quiere cambiar el color de acento de azul a verde, el cambio es una sola línea:

`ACCENT = "#2E7D32"` . Sin tokens, habría que buscar y reemplazar `#2962FF` en 40+ lugares.

Paleta de tipos Pokémon

Colores canónicos del universo Pokémon (usados en los juegos y en Bulbapedia). Darles el color correcto no es decorativo — el lector que conoce Pokémon los reconoce inmediatamente, lo que reduce la carga cognitiva del dashboard.

Normal	Fire	Water	Electric	Grass	Ice
Fighting	Poison	Ground	Flying	Psychic	Bug
Rock	Ghost	Dragon	Dark	Steel	Fairy

Tipografías

El dashboard usa dos fuentes con propósitos distintos:

Fuente	Dónde se usa	Por qué
Press Start 2P	Brand, logo, títulos de página, nav	Fuente "gamer/pixel" — evoca Pokémon directamente
Inter	Cuerpo de texto, datos, labels	Diseñada para pantallas, legible a tamaños pequeños

CSS · REGLA BASE

```
* { font-family: 'Inter', sans-serif !important; }
.pokemon-brand { font-family: 'Press Start 2P', monospace !important; }
```

Escala establecida en el sistema de diseño. Press Start 2P se usa en una escala reducida porque visualmente es una fuente muy "grande".

Press Start 2P

Brand / logo	0.85rem	Header principal
Título de página h2	0.80rem	.page-header h2
Items de navegación	0.58rem	Sidebar radio buttons
Subtítulo principal	0.55rem	.page-header p
Labels sidebar	0.52rem	st.caption()

Inter

KPI value	1.90rem	.kpi-value
Section title	1.05rem	.section-title
Narrative card title	0.90rem	Sección ETL
Nombre en poke-card	0.82rem	Galería
Selectbox value	0.82rem	Filtro burbuja
KPI label	0.75rem	.kpi-label
Type badge / bst label	0.72rem	Galería
Hero badge	0.72rem	Vista General
Winner name (Comparación)	0.68rem	Ganador en comparación
Type badge Comparación	0.68rem	Comparación
Selectbox label	0.62rem	Filtro burbuja
Labels varios	0.56rem	Sidebar, detalles

El problema: Streamlit no expone sus estilos

Streamlit renderiza los componentes con React internamente. No existe un archivo CSS que editar directamente. La única forma de personalizar los estilos es:

PYTHON

```
st.markdown("""
<style> ... </style>
""", unsafe_allow_html=True)
```

`unsafe_allow_html=True` es necesario porque Streamlit por defecto escapa el HTML para evitar XSS. Al usarlo, asumimos la responsabilidad de que el HTML es seguro.

Cómo apuntar a componentes nativos

Streamlit añade atributos `data-testid` a sus contenedores. Se pueden usar como selectores CSS:

CSS

```
/* Sidebar */
[data-testid="stSidebar"] { background: ...; }

/* Gráfica Plotly */
[data-testid="stPlotlyChart"] { border-radius: 20px; }

/* Selectbox completo (label + dropdown) */
.main div[data-testid="stSelectbox"] { background: ...; }
```



Advertencia de versión

Los `data-testid` pueden cambiar entre versiones de Streamlit. Si el CSS deja de funcionar tras una actualización, lo primero es inspeccionar el DOM y verificar que el atributo sigue siendo el mismo.

Scoping — .main vs sidebar

CSS

```
/* CORRECTO: solo afecta selectboxes en el área principal */
.main div[data-testid="stSelectbox"] { ... }

/* INCORRECTO: afectaría también los selectboxes del sidebar */
div[data-testid="stSelectbox"] { ... }
```

Por qué envolver widgets en <div> NO funciona

PYTHON · ANTIPATRÓN

```
st.markdown('<div class="mi-contenedor">')
st.selectbox("Filtro", options=[...])    # ← Este widget NO queda dentro del div
st.markdown('</div>')
```

Cada llamada a `st.markdown()` y `st.selectbox()` genera un elemento DOM separado. La única solución para aplicar estilos a un widget nativo es apuntar directamente a su `data-testid` con CSS global.

KPI Card

Las KPI cards son HTML generado dinámicamente en Python:

PYTHON

```
def kpi_card(label: str, value: str, sub: str = "") → str:
    return f"""
        <div class="kpi-card">
            <div class="kpi-label">{label}</div>
            <div class="kpi-value">{value}</div>
            { "<div class='kpi-sub'" + sub + "</div>" if sub else "" }
        </div>
    """
```

El CSS de `.kpi-card` define la identidad visual del componente:

- `border-left: 4px solid ACCENT` — la franja azul característica
- `border-radius: 16px` — esquinas redondeadas
- `animation: fadeUp 0.5s` — aparición con slide-up

Type Badge

PYTHON

```
def type_badge(type_name: str) → str:
    color = TYPE_COLORS.get(type_name, "#90A4AE")
    return f'<span class="type-badge" style="background:{color};">{type_name}</span>'
```

Hero Container

Bloque central de Vista General cuando se selecciona un Pokémon destacado:

CSS

```
.hero-container {
    background: linear-gradient(135deg, #EEF2FF 0%, #E8F4FD 100%);
    border-radius: 24px;
    min-height: 300px;
    animation: slideIn 0.6s ease both;
}
```

Gradiente suave azul-lila a azul claro consonante con el acento. La animación `slideIn` lo hace aparecer desde la izquierda, diferenciándolo de los `fadeUp` del resto.

Poke Card (Galería)

CSS

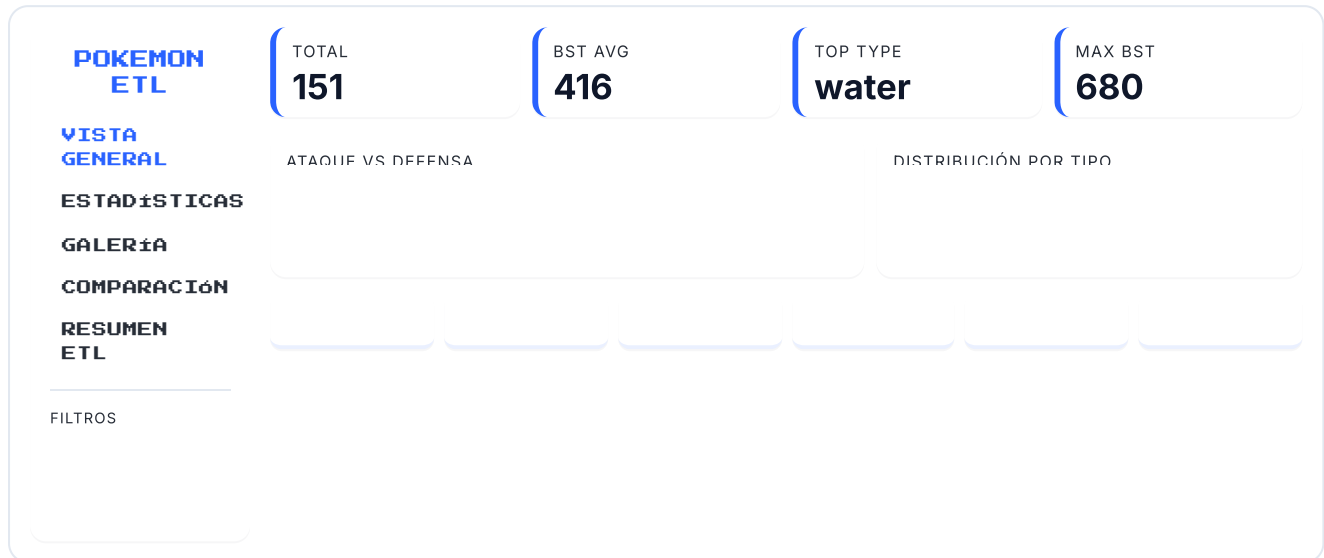
```
.poke-card {  
  transition: transform 0.2s ease, box-shadow 0.2s ease;  
}  
.poke-card:hover {  
  transform: translateY(-6px);  
  box-shadow: 0 8px 24px rgba(41,98,255,0.15);  
}
```



Solo un componente con hover

La poke card es el único elemento con interacción hover en el dashboard — refuerza visualmente que la Galería es un *explorador* y no solo una tabla.

Estructura del sidebar



Radio buttons como navegación

Streamlit no tiene un componente Navigation nativo. El patrón estándar es usar `st.radio()` en el sidebar. El CSS oculta el indicador del radio y estiliza los labels como ítems de menú:

CSS

```
section[data-testid="stSidebar"] div[role="radiogroup"] label {
  padding: 0.5rem 0.75rem;
  border-radius: 10px;
  font-family: 'Press Start 2P', monospace !important;
  font-size: 0.58rem !important;
  color: #555555;
  transition: background 0.15s ease, color 0.15s ease;
}
section[data-testid="stSidebar"] div[role="radiogroup"] label:hover {
  background: rgba(41,98,255,0.08);
  color: #2962FF;
}
section[data-testid="stSidebar"] div[role="radiogroup"] input[type="radio"] {
  display: none; /* Oculta el círculo del radio button */
}
```

El logo en base64

PYTHON

```
_logo_path = Path(__file__).parent / "assets" / "Font-pokemon.png"
with open(_logo_path, "rb") as f:
    logo_b64 = base64.b64encode(f.read()).decode()

st.sidebar.markdown(f'', unsafe_allow_html=1)
```

i ¿Por qué base64 y no `st.image()`?

`st.image()` es más simple pero no permite control fino de posicionamiento CSS. Al convertir la imagen a base64 e incrustarla en un `<img>` HTML, se puede aplicar cualquier estilo CSS directamente.

Vista General

[tab_overview\(\)](#)

Propósito: Primera impresión del dataset. KPIs globales + gráficas de distribución + Pokémon destacado.

- **4 KPI cards** — Total Pokémon, BST promedio, tipo más común, Pokémon con mayor BST.
- **Filtro burbuja** — Selectbox con CSS de card para seleccionar un Pokémon destacado.
- **Hero container** — Sprite oficial + stats del Pokémon seleccionado (o mensaje si se elige "Todos").
- **Gráficas** — Scatter Ataque vs Defensa coloreado por tipo + Distribución de tipos (bar chart).

Estadísticas

[tab_stats\(\)](#)

Propósito: Análisis profundo de las distribuciones de stats.

- Histograma de BST con línea de media.
- Boxplot de BST por tipo primario.
- Heatmap de correlación entre estadísticas.
- Top 20 por BST (bar chart horizontal).

Galería

[tab_gallery\(\)](#)

Propósito: Explorador visual de todos los Pokémon. Permite buscar y ver la colección.

- Poke cards en grid (4 por fila) con sprite oficial, nombre, tipos y BST.
- Las cards respetan los filtros del sidebar.
- Hover effect en cada card.

Comparación

[tab_compare\(\)](#)

Propósito: Comparar dos Pokémon head-to-head con radar chart y detalle de stats.

- Dos selectboxes (Pokémon A vs Pokémon B).
- Radar chart con `go.Scatterpolar` superpuesto (dos trazas, una por Pokémon).
- Tabla de stats con ganador resaltado en cada estadística.
- Badge de ganador global (mayor BST).

Resumen ETL

`tab_etl()`

Propósito: Documentar el pipeline directamente en el dashboard.

- Diagrama de flujo Mermaid renderizado como HTML.
- Narrative cards: ¿Por qué este proyecto? / ¿Qué hace el pipeline? / ¿Cómo está construido?
- Fases ETL con íconos y descripciones técnicas.
- Tabla del dataset (columnas, tipos, descripción).

st.plotly_chart() — La función principal

PYTHON

```
st.plotly_chart(fig, use_container_width=True)
```

`use_container_width=True` hace que la gráfica tome el 100% del ancho disponible. Sin esta opción, Plotly usa su tamaño por defecto (700px fijo).

Plotly Express vs Plotly Graph Objects

	<code>plotly.express</code> (px)	<code>plotly.graph_objects</code> (go)
Sintaxis	Una línea	Más verbosa
Cuándo usar	Gráficas estándar	Control fino, gráficas compuestas
Ejemplo	<code>px.scatter(df, x="attack", y="defense")</code>	<code>go.Scatter(x=..., y=...)</code>

El dashboard usa **ambos**: Express para gráficas simples (Vista General y Estadísticas), Graph Objects para el radar chart de Comparación.

Radar Chart (Comparación)

PYTHON

```
fig = go.Figure()
for poke, color in zip([poke_a, poke_b], [ACCENT, "#E65100"]):
    fig.add_trace(go.Scatterpolar(
        r=stats_values,
        theta=stat_labels,
        fill="toself",
        name=poke["name"],
        line=dict(color=color),
    ))
fig.update_layout(polar=dict(radialaxis=dict(range=[0, max_val])))
```

`go.Scatterpolar` es el tipo de gráfica para radar/spider charts. `fill="toself"` rellena el área bajo la línea, lo que hace visualmente intuitiva la comparación de áreas.

CSS de las gráficas Plotly

CSS

```
[data-testid="stPlotlyChart"] {  
  background: #FFFFFF;  
  border-radius: 20px;  
  padding: 0.6rem 0.8rem 0.4rem;  
  box-shadow: 0 4px 18px rgba(0,0,0,0.07);  
}
```



Consistencia visual

El contenedor de cada gráfica Plotly recibe una card visual con esquinas redondeadas y sombra. Todas las gráficas lucen como componentes del mismo sistema.

@st.cache_data — Por qué es crítico

PYTHON

```
@st.cache_data
def load_data() → pd.DataFrame:
    with sqlite3.connect(DB_PATH) as conn:
        return pd.read_sql("SELECT * FROM pokemon", conn)
```

Sin `@st.cache_data`, cada interacción del usuario re-ejecutaría la consulta SQL y re-leería el archivo. Con el decorador, Streamlit guarda el resultado en memoria — la lectura SQL ocurre una sola vez.

Fallback a CSV

PYTHON

```
@st.cache_data
def load_data() → pd.DataFrame:
    try:
        with sqlite3.connect(DB_PATH) as conn:
            return pd.read_sql("SELECT * FROM pokemon", conn)
    except Exception:
        return pd.read_csv(CSV_PATH)
```

Si `pokemon.db` no existe, la app carga directamente desde el CSV procesado. El dashboard funciona sin necesidad de ejecutar el ETL completo.

Filtros del sidebar

PYTHON

```
tipos      = st.sidebar.multiselect("Tipos", options=sorted(df["type_primary"].unique()))
rango_bst  = st.sidebar.slider("BST", min_value=0, max_value=int(df["bst"].max()))
tiers      = st.sidebar.multiselect("Tier de poder", options=...)

df_filtered = df.copy()
if tipos:
    df_filtered = df_filtered[df_filtered["type_primary"].isin(tipos)]
df_filtered = df_filtered[df_filtered["bst"].between(*rango_bst)]
if tiers:
    df_filtered = df_filtered[df_filtered["power_tier"].isin(tiers)]
```

El DataFrame original `df` se mantiene sin modificar — se usa para el comparador (que debe poder acceder a todos los Pokémon, independientemente de los filtros).

CSS · FADEUP

```
@keyframes fadeUp {  
  from { opacity: 0;  
        transform: translateY(16px); }  
  to   { opacity: 1;  
        transform: translateY(0); }  
}
```

CSS · SLIDEIN

```
@keyframes slideIn {  
  from { opacity: 0;  
        transform: translateX(-20px); }  
  to   { opacity: 1;  
        transform: translateX(0); }  
}
```

fadeUp se aplica en `.card`, `.kpi-card` y `.poke-card` — crea la sensación de que la página "se construye" al cargarse. **slideIn** se usa en `.hero-container` — su entrada desde la izquierda lo distingue como protagonista de la Vista General.

**Regla de duración**

0.15 s para transiciones de hover/menú es suficiente para sentirse responsivo sin ser llamativo. Las transiciones más lentas (> 300 ms) hacen que la UI parezca lenta.

Ejecutar el dashboard

BASH

```
# Desde la raíz del proyecto
$ streamlit run app.py
```

El dashboard se abre automáticamente en `http://localhost:8501`. No requiere ejecutar el pipeline ETL primero — los datos ya están en `data/`.

Configuración `.streamlit/config.toml`

TOML

```
[server]
headless = true
```

`headless = true` evita que Streamlit abra el navegador automáticamente en entornos de servidor. En desarrollo local, omitir esto y dejar que Streamlit abra el navegador.

Cambiar la paleta de colores

PYTHON · APP.PY

```
ACCENT    = "#2962FF"    # ← Cambiar aquí para nuevo color de acento
BG        = "#F5F7FA"    # ← Fondo general
CARD_BG   = "#FFFFFF"    # ← Fondo de cards
TEXT      = "#111111"    # ← Color de texto principal
TEXT_SUB  = "#555555"    # ← Texto secundario
```

El CSS usa estas constantes mediante f-strings (`f"color: {ACCENT}"`), por lo que el cambio se propaga a todo el dashboard.

Añadir una nueva página

1. Crear la función `tab_nueva_pagina(df: pd.DataFrame) → None`.
2. Añadir el nombre a `NAV_PAGES`.
3. Añadir el caso al router en `main()`.

PYTHON

```
NAV_PAGES = ["Vista General", "Estadísticas", "Galería",  
             "Comparación", "Resumen ETL", "Nueva Página"]  
  
# ... dentro de main():  
elif page == "Nueva Página":  
    tab_nueva_pagina(df_filtered)
```

Dependencias necesarias

BASH

```
$ pip install streamlit plotly pandas
```

El dashboard no requiere las dependencias del pipeline ETL (`requests` , `tenacity` , `loguru`) para funcionar.

Fin del documento

Manual Visual — Dashboard Streamlit Pokémon · v1.0

© 2026 · Documento de referencia interna
Python · Streamlit · Plotly · CSS